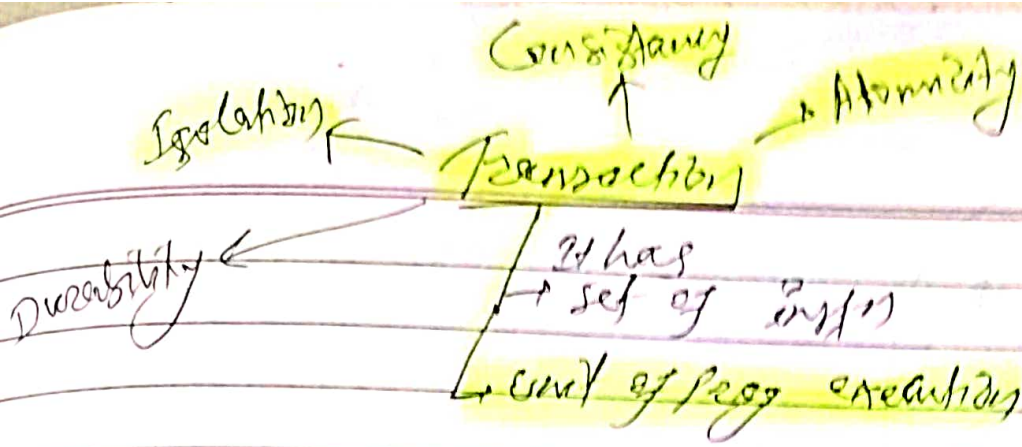
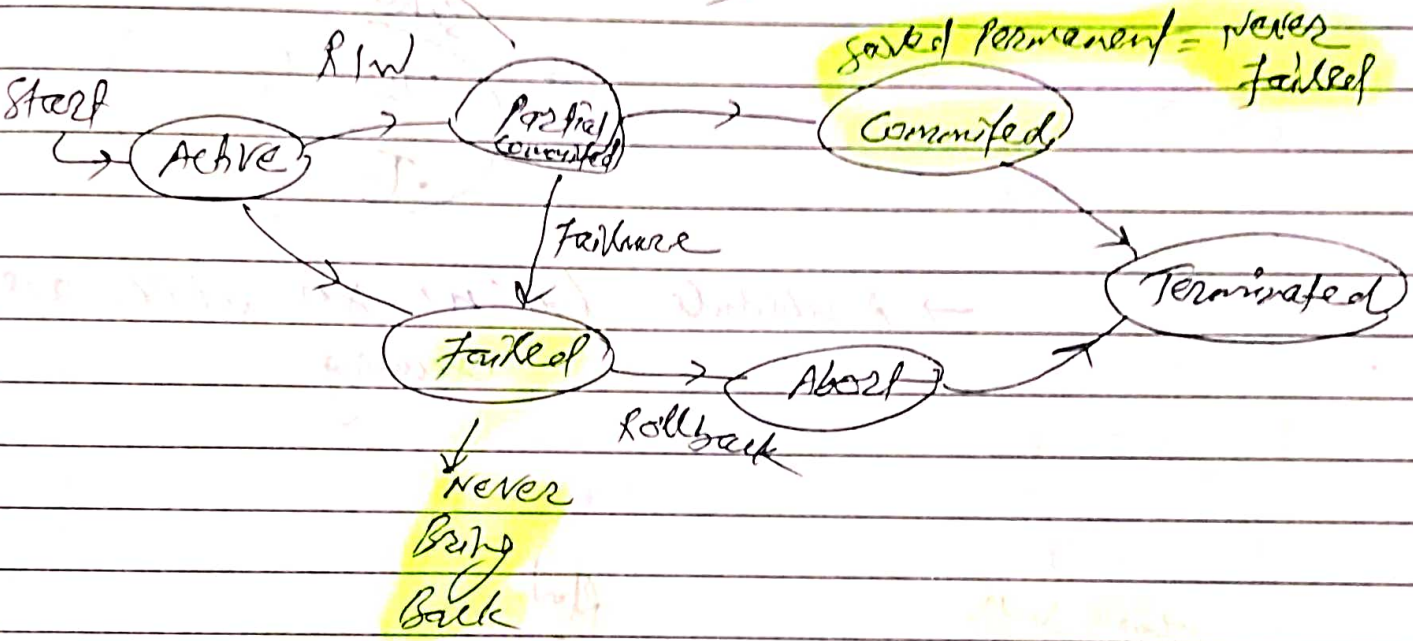
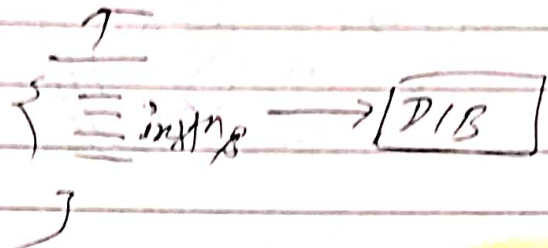


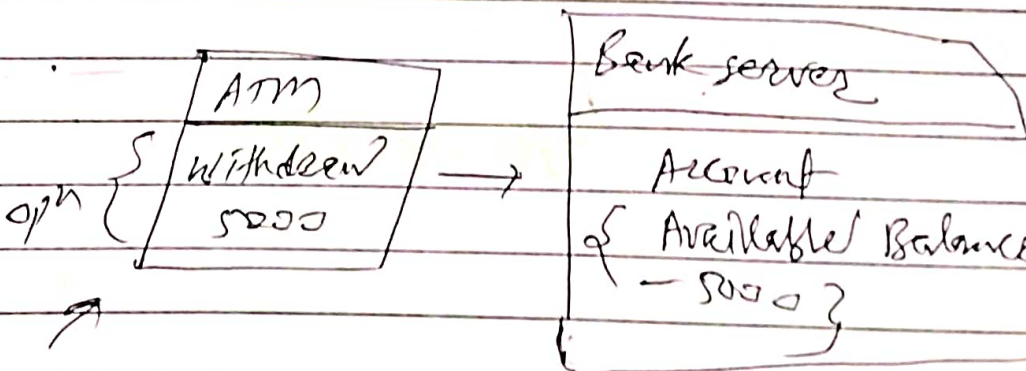


not committed till not
partial - committed



* Atomicity → Either entire transaction takes place or no transaction happen at all.

→ Partial execution is not allowed of transaction



Either no withdraw,

→ if withdraw then must updated at Account

* Consistency $\Rightarrow$ system must go to valid state after each transactions.

State-1

A 1000

B 2000

Addⁿ of Both {A+B}
3000

now, if 500 transfer from A to B,

system must be in consistent state,

State-2

A 1000 B 2000 + 500
500 500

Final state

A 500 B 2500

Addⁿ of Both {A+B}
= 3000

* Isolation $\Rightarrow$ Each transaction should be isolated to each other.

T₁ A 2000 $\rightarrow$ only one txn allowed to operate on data at a time

T₂ X A 2000 $\rightarrow$ multiple txn not affect single data at a time
X T₃ X

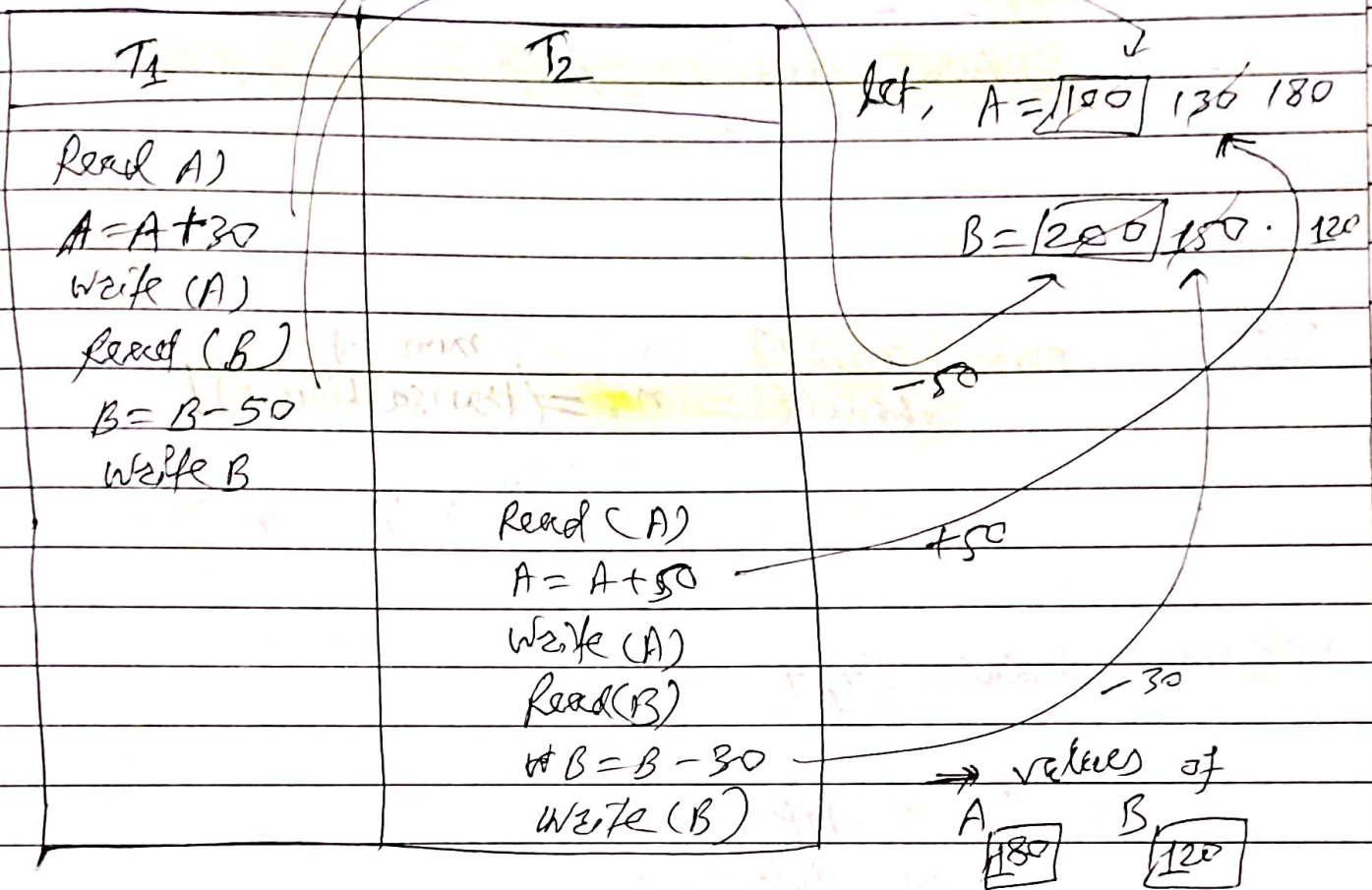
* Durability $\Rightarrow$ Transaction $\leftarrow$ Commit $\rightarrow$ permanent guaranteed
fail $\Rightarrow$ it should be fail

Each x^n perform their execution one after another

Page No. _____

Date: / /

* serial schedule



drawback of serial schedule

① more waiting time $\left\{ T_1, T_2, \dots, T_n \right.$
wait till T_n

② Resource utilization less

any data/resource

can't utilized by other

when one x^n waiting if

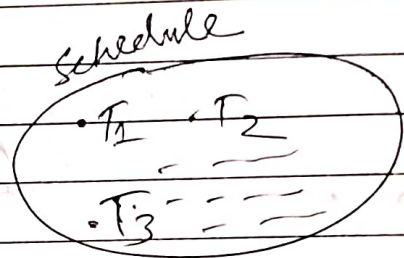
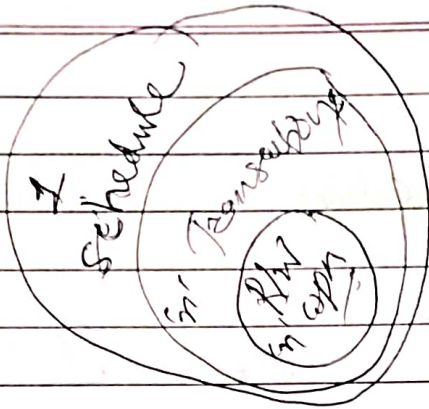
③ less throughput

larger x^n take more time,
hence smaller x^n wait

④ late response

$\therefore$ num x^n Executing per time unit = less

small x^n has to wait, until current one is Executing.



→ A schedule has 'n' x^n which are executing

for x^n

Throughput = num (operations) perform per time unit.

↑

R/W

Throughput = num (Transaction) completed per time unit

for schedule

Can be write in any way

(multiple way to write Concurrent)

Page No. Schedule

Date: 1/1

* Concurrent Schedule (or Non-serial schedule)

one of
the
way
of
concurrent
schedule

T ₁	T ₂	let, A = 100 130 180
Read (A)		
A = A + 30		
Write (A)		+50
	Read (A)	
	A = A + 50	B = 100 150 120
	Write (A)	-50
Read (B)		
B = B - 50		
Write (B)		-30
	Read (B)	
	B = B - 30	
	Write (B)	

Final value of

A	B
<u>180</u>	<u>120</u>

Benefits

- T₂ get response fast, waiting time reduced.
- A's final value immediately computed

→ Blind
~~statement~~ write

↳ without reading, perform writing opⁿ

Note $\Rightarrow$ $\text{num}(\text{serial})$ $\text{schedule} = n! = \frac{\text{num of}}{(\text{transactions})!}$

{ for $T_1, T_2, \dots, T_n$ x }

Note $\Rightarrow$ schedule (T_1, T_2, T_3)

$T_1 = x$ statements

$T_2 = y$ statements

$T_3 = z$ statements

$\therefore \text{num}(\text{concurrent schedule}) = (x+y+z)!$

$x! * y! * z!$

Note $\Rightarrow$ $\text{Num}(\text{non-serial schedules}) = \frac{\text{num}(\text{concurrent schedule})}{\text{num}(\text{serial schedule})}$

Note $\Rightarrow$ $\boxed{\text{Total schedules} \approx \text{concurrent schedule} + \text{serial schedule}}$

Another name of total schedule

concurrent schedule = Total schedule = serial + non serial

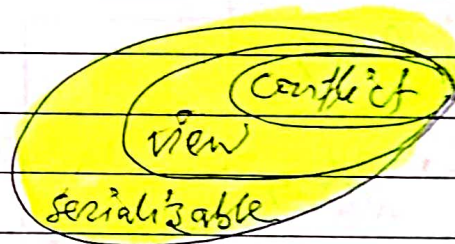
* Serializable schedule $\rightarrow$ checking for individual schedule

$\rightarrow$ if the o/p of concurrent schedule same as serial schedule,
 $\therefore$ called as serializable schedule

$\Rightarrow$ To check schedule is serializable or not. ?

- ① Conflict serializable
 - ② View serializable
- } 2 techniques to check

checking for individual schedule (XN)



① Conflict serializability

$\rightarrow$ uses precedence graph

$\rightarrow$ If precedence graph = Acyclic $\{ \text{Not contain any cycle} \}$

$\therefore$ schedule = conflict serializable

$\rightarrow$ It is the highest serializability condition

if any schedule has conflict serializability, then it is view serializable as well

* How to design precedence graph?

T_1	T_2
$R(A)$	
	$R(A)$

$(T_1) \quad (T_2)$

no edge

Read $\nrightarrow$ Read
no edge

T_1	T_2
$R(A)$	$R(A)$
	$W(A)$

$(T_1) \rightarrow (T_2)$

Read $\rightarrow$ Write edge

T_1	T_2
$W(A)$	
	$R(A)$

$(T_1) \rightarrow (T_2)$

write $\rightarrow$ Read edge

T_1	T_2
$W(A)$	
	$W(A)$

$(T_1) \rightarrow (T_2)$

write $\rightarrow$ write edge

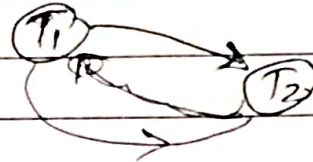
Edges within $\times^n$ not needed

Page No. _____

Date: ____/____/____

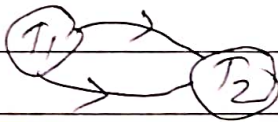
Ex 8 $\rightarrow$ check for conflict serializable or not?

δ	T_1	T_2
	$R(A)$	
	$W(A)$	
		$R(A)$
		$W(A)$
	$R(B)$	
	$W(B)$	
		$R(B)$
		$W(B)$



$\rightarrow$ Edge not drawn on δ^n
R/W within $\times^n$

$\rightarrow$ Edge not drawn on δ^n
on different data item



Acyclic = no cycle

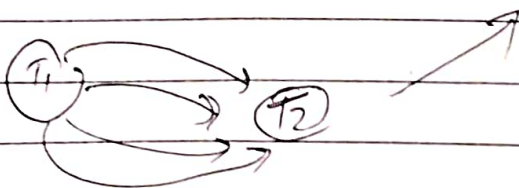
$\therefore$ Conflict schedule

$\rightarrow$ direction of all edges
from T_1 to T_2

$\therefore$ only one edge
can be drawn,

not need to draw

multiple edges
towards / of same transaction

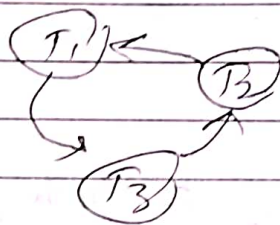


Edges (Read $\rightarrow$ Read) not possible

Page No. _____
Date: ____/____/____

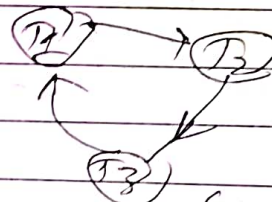
Ex $\rightarrow$

T_1	T_2	T_3
$R(x)$		
$W(x)$	$R(x)$	
		$R(x)$
	$W(x)$	



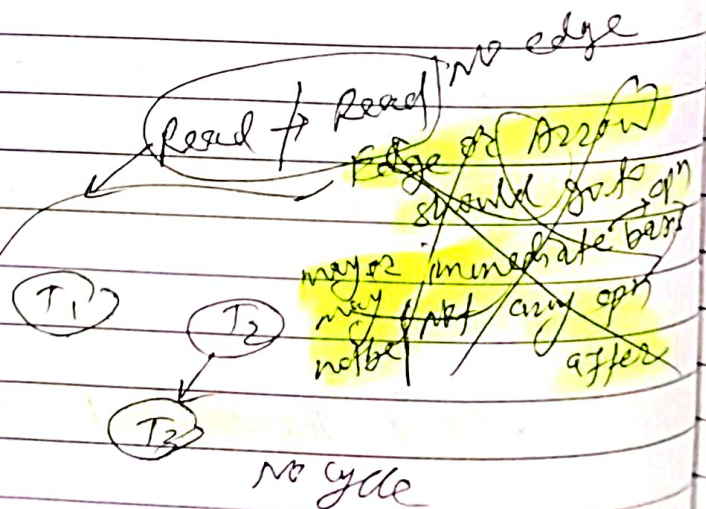
cycle, $\therefore$ Not conflict serializable

T_1	T_2	T_3
	$R(x)$	
$R(x)$	$W(x)$	
		$R(x)$
$W(x)$		



cycle, $\therefore$ Not serializable

T_1	T_2	T_3
	$R(x)$	
	$W(x)$	
$R(x)$		$R(x)$
$W(x)$		



$\therefore$ conflict serializable

$\therefore$ As conflict $\rightarrow$ view as well.

more than one Any cyclic schedule is also a conflict equivalent Page No. _____

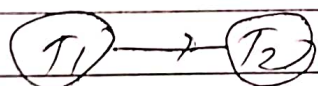
* Conflict Equivalent Schedules $\Rightarrow$ for multiple schedule checking

$\rightarrow S_1$ conflict equivalent to S_2 ,

only if Precedence graph of both schedule is same.

S_1

T_1	T_2
R(A)	
W(A)	
	R(A)
	X - W(A)
R(B)	X
W(B)	
	R(B)
	W(B)



S_2

T_1	T_2
R(A)	
W(A)	
R(B)	
	W(A)
	R(A)
W(B)	
	R(B)
	W(B)



Both $S_1 \equiv S_2$ conflict
Equivalent

Page No. _____

Date: ____/____/____

⇒ If there are 2 schedules & both are cyclic then

They might be conflict equivalent.

* Testing on Remaining $\times^n$ which is not conflict serializable

* View Serializable

- when a schedule is not conflict then check for view
- iff it is equivalent to its serial schedule

checking View

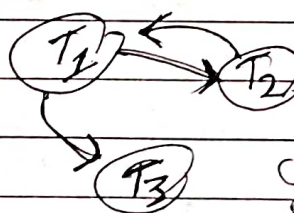
① Both ~~has~~ has same $\times^n$ Schedules

②

Data item	Initial Read	Write-Read Sequence	Final Write

Ex $\Rightarrow$

	T_1	T_2	T_3
R(A)			
		W(A)	
W(A)			
			W(A)



Cyclic

$\therefore$ Not conflict

check view

Data item	Initial Read	W/R Sequence	Final write
A	T_1	$T_1 + T_2 + T_3$	T_3

Fairly condition

$\therefore$ not view serializable.

As T_1 Read initially must write finally

Note:

$\left\{ \begin{array}{l} \text{Not conflict} \\ \text{serializable} \end{array} + \begin{array}{l} \text{Not Blind} \\ \text{write} \end{array} \right\} = \text{Not View} \\ \text{serializable}$

Note:

$\left\{ \begin{array}{l} \text{Not conflict} \\ \text{serializable} \end{array} + \begin{array}{l} \text{Blind} \\ \text{write} \end{array} \right\} = \text{may or may} \\ \text{not be view} \\ \text{serializable}$

Check
view

$\Rightarrow$ specific Data items Read by
which
serializable
concln
that + n should perform final write.

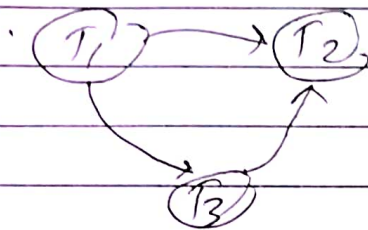
multiple order possible
 → performance

Page No. _____

Date: ____/____/____

* Topological order

used to find order of equivalent of conflict serializable



$T1 \rightarrow T3 \rightarrow T2$ is serial order

$T1 \rightarrow T2 \rightarrow T3$ is not order

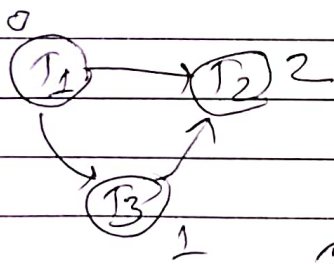
→ Compute Indegree of each node

use Rule

→ Remove Indegree '0' node then Indegree '1' then till last.

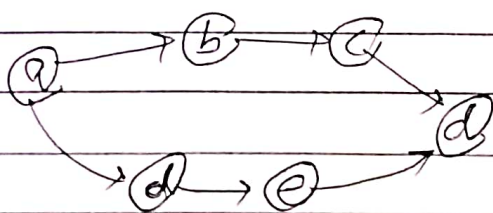
while removing node order of increasing sequence

must follow $\{0, 1, 2, \dots\}$



$T1 \rightarrow T3 \rightarrow T2$
 Indeg(0) Indeg(1) Indeg(2)

Ex → find num (topological) ordering possible?



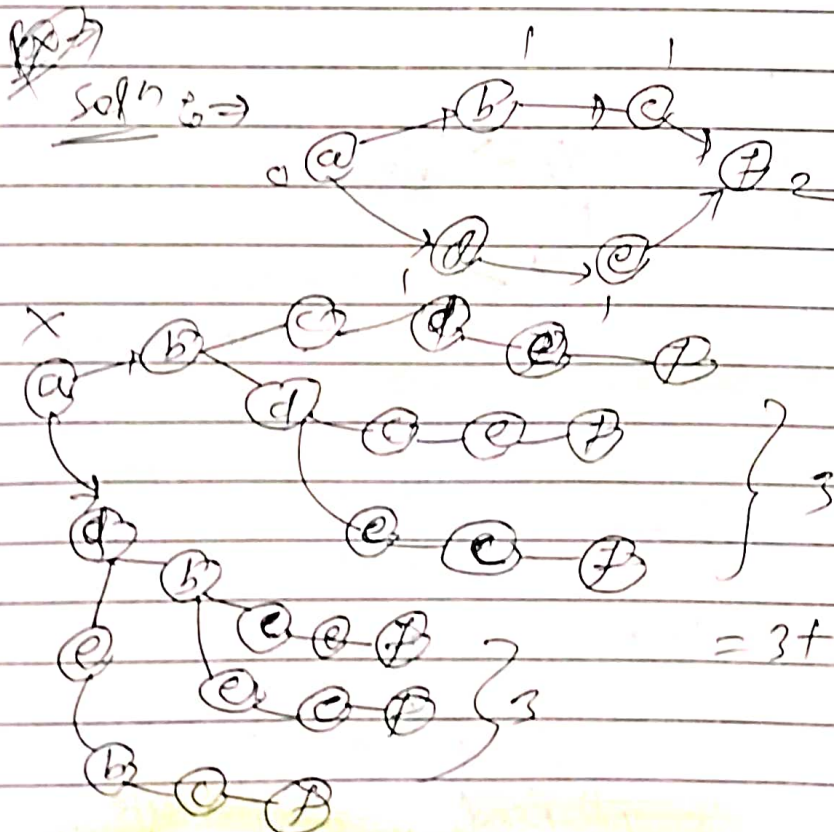
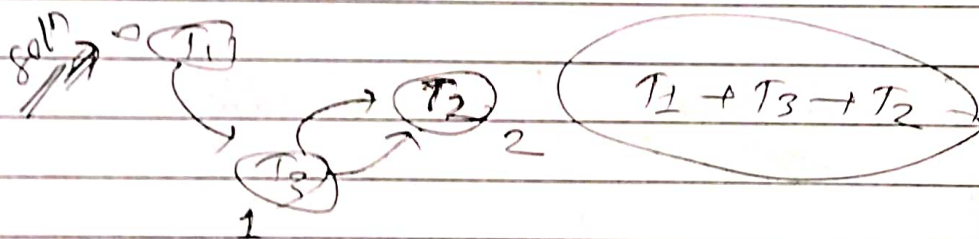
→ must contain All vertex

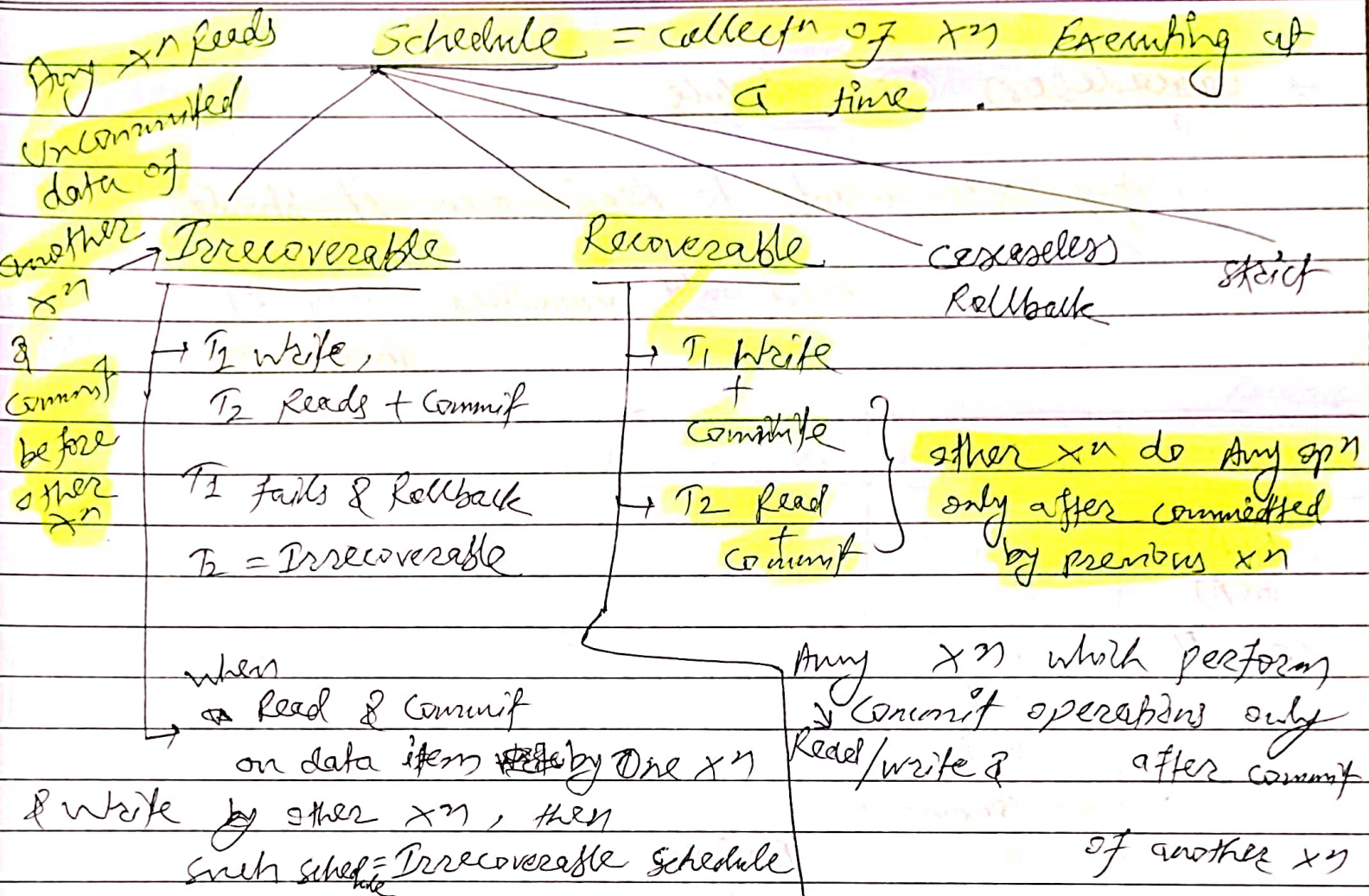
→ There must a path exists of each vertex.

Ex $\Rightarrow$ Find correct order of serializability?

T_1	T_2	T_3
$R(x)$	$R(x)$	$R(y)$
$w(x)$	$w(x)$	$w(x)$
	$R(x)$	
	$w(x)$	

look Indegree of each node at precedence graph
start with node (x) has least degree or 0 deg then go in increasing order to form sequence.





T_1	T_2
R(A)	
W(A)	
	R(A)
	Commit
R(B)	
--	
Fails & Rollback	

T_1	T_2
R(A)	
W(A)	
Commit	
	R(A)
	Commit

Schedule (S) = Recoverable

RAW dependency
Schedule S = Irrecoverable because of T_2 can't be recover.

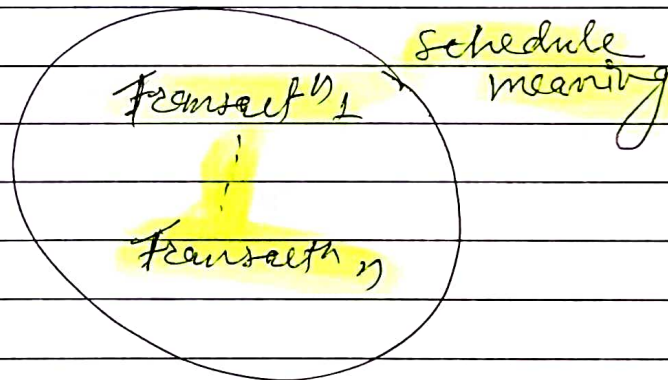
Schedule Types

⇒ All Cascadeless schedule are Recoverable schedule also But,

⇒ Not All Recoverable schedules are Cascadeless.

Note

⇒ If In any schedule, Any one x_n is Recoverable than entire schedule is Recoverable.



Recoverable schedule ⇒ when All x_n in schedule are Recoverable, then schedule is Recoverable.

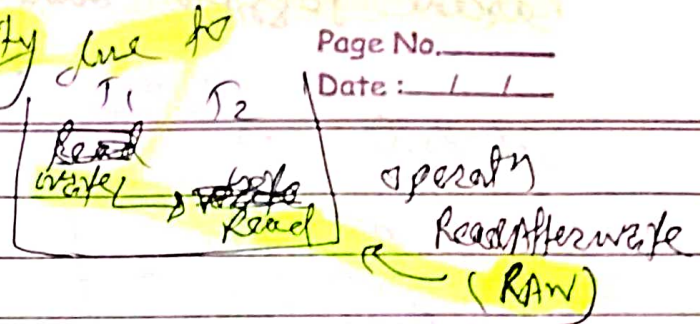
Recoverable x_n ⇒ By Read/Write, x_n should perform commit operation anytime then it is Recoverable.

Cause Irrecoverability due to

Page No. _____

Date: ____/____/____

* Cascade Rollbacking

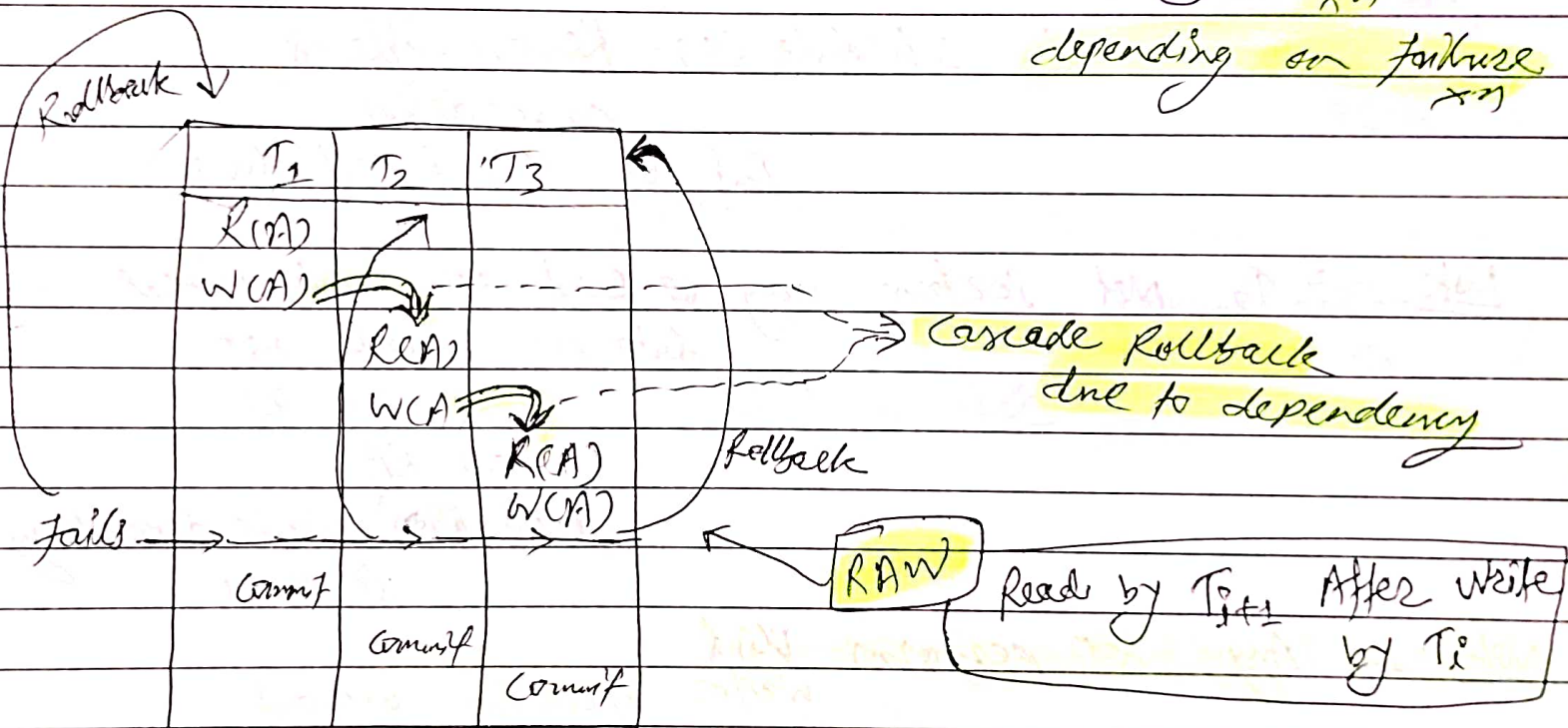


If 'n' num of x^n runs & Because of any one x^n , all 'n' x^n needs to Rollback failure

Such condⁿ / Rollback = Cascade Rollback

If causes wastage of time of Rollbacking all other x^n

depending on failure x^n



Ans schedule S = Recoverable, But not

cascadeless schedule. It has cascading Rollback effect.

Note $\Rightarrow$ A Recoverable schedule may have cascading Rollback

A Recoverable schedule may not be cascadeless schedule

* Cascadeless Trans schedule

↳ Any x^n wants to Read then it should

Read only committed data of another x^n .

T_1	T_2	T_3
R(A)		
w(A)		
Commit		
	R(A)	
	w(A)	
	Commit	
		Read(A)
		w(A)
		Commit

Schedule $S = \{T_1, T_2, T_3\}$ Transactions

Is schedule S = Cascadeless schedule.

Is recoverable

Recoverable
Cascadeless
schedule

R/W only on committed data

Page No. _____

Date: ____/____/____

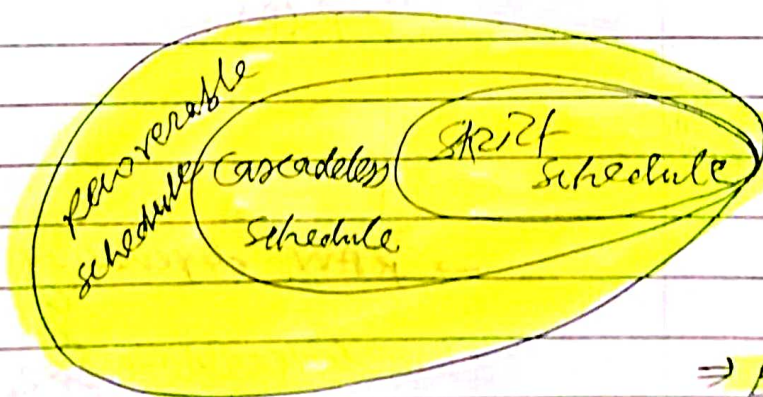
* strict schedule

→ A x_n Neither allowed to read or write on data items until the last x_n has written it is committed or aborted.

T_1	T_2
$W(A)$	
Commit or Rollback	
	$R(A)$ $W(A)$

} T_2 will do Read/Write only after Commit/Rollback by T_1 .

Schedule (S) = strict schedule



⇒ Every strict schedule is
Cascadeless schedule
& Recoverable

But vice versa Not possible

Cascade possible only on RAW (on Read on Uncommitted Write of Previous Tx)

Page No.

Date:

Schedule (S)

T ₁	T ₂
R(A)	
W(A)	
Commit	W(A) // uncommitted write

Cascadeless & Recoverable

T₂ perform blind write

∴ Schedule (S) = Recoverable & cascadeless

But not strict schedule

Note ⇒ T₂ not perform any read on uncommitted data so cascade not possible as no RAW

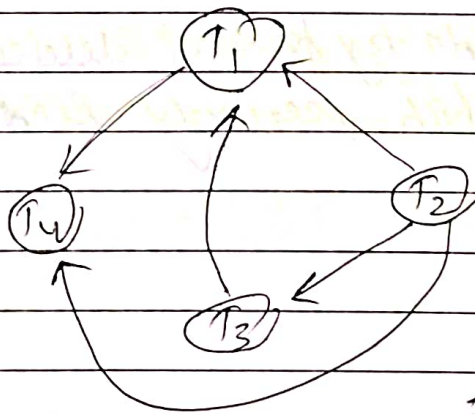
Read After Write dependency is there

Note ⇒ Any Tx perform blind write operation without

Read ↓ "can never" cause cascade & dependency RAW on uncommitted data

Ex $\Rightarrow$

T_1	T_2	T_3	T_4
	$R(x)$		
		$W(x)$ Commit	
$W(x)$ Commit			
	$W(x)$ $R(z)$ Commit		
			$R(x)$
			$R(y)$
			Commit

check for
schedule isRecoverable and
conflict
serializable
or not?solnno cycle, $\therefore$ conflict
serializable~~no edge from $T_1 \rightarrow T_2$~~
as data items are
different. $\Rightarrow$ Commit operability by each x^n who performs write opn
 $\therefore$ Recoverable $\therefore$ schedule (S) = conflict serializable &
Recoverable.

T_1	T_2
$R(A)$	
	$R(A)$

No $\rightarrow$ Irrecoverable = Recoverable
 Problem of dependency

RAR

T_1	T_2
$R(N)$	
	$w(N)$

No $\rightarrow$ Irrecoverable = Recoverable
 Problem of dependency

Blind write $\rightarrow$ cause no cascade problem

WAR

T_1	T_2
$w(A)$	
Commit	
	$R(A)$

No $\rightarrow$ Irrecoverable = Recoverable
 No cascade Rollback
 as read on committed data

RAW = no effect due to commit

T_1	T_2
$w(A)$	
	$R(A)$

$\rightarrow$ RAW dependency
 $\rightarrow$ Irrecoverable error
 $\rightarrow$ It is dirty read

$\rightarrow$ Read uncommitted data

* Concurrency Problems

- ① Dirty Read Problem = Reading uncommitted data
 ↳ Cause Irrecoverable error

Rollback

T ₁	T ₂
R(A)	
W(A)	
	R(A)
	Commit
Fail	

- ② Un Repeatable Read Problem

↳ Any Transactn Read data item more than one time & every time it found a change in data item

Initially
A=5

T ₁	T ₂
R(A)	
R(A)	

A=10

T ₁ R(A) = 5, \$
T ₂ W(A) = 10
T ₁ R(A) = 10 ←

• T₁ confused for multiple / changing values of A.

③ Lost update problem

- write without reading = blind write
- overwriting on data item without reading

T_1	T_2
$R(A)$	
$W(A)$	
	$W(A)$

override
 $T_2 \Rightarrow$ overwrite the A's value
 which write by T_1
 $\therefore T_1$ lost its update

④ Phantom Read → Transactn try to read deleted data item which previously read successfully.

T_1	T_2
$R(A)$	
	$R(A)$
$Delete(A)$	
	$R(A)$

$T_2 =$ Confused, as already has read A's value

now, no value of A on Read (A)

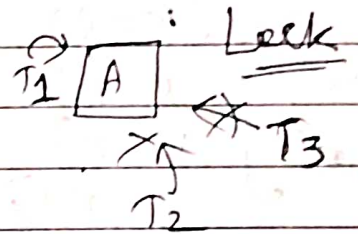
* locking Protocol → used to achieve Concurrency serializability

Types of lock

used to update data value

→ shared lock = $S(A)$
(only Read)

→ Exclusive lock = $X(A)$
(Both R/W)



→ When one x^n operate on data A, no other will interfere.

	$S(A)$	$X(A)$
$S(A)$	yes	NO
$X(A)$	NO	NO

⇒ when A x^n operate shared lock $S(A)$,

multiple shared lock allowed on same data

can other x^n used shared lock on same data item ($S(A)$) = yes/Allowed

⇒ If shared lock on data A, not $X(A)$

NO Exclusive lock Allowed on data.

⇒ For Exclusive lock on A, not any other lock perform on data

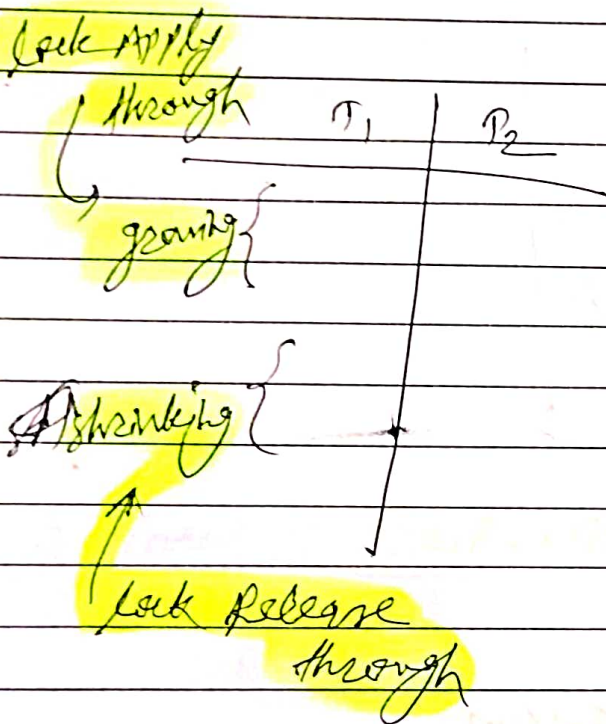
⇒ Once Release the lock, Again to Acquire lock on ~~that~~ data item of the (xn) ~~is~~ not allowed

Page No.
Date: / /

* 2 Phase locking plf -
 growing phase
 shrinking phase

1st growing phase
then
shrinking phase

After shrinking phase
~~it is starting~~ then
growing phase
not Allowed



→ Once lock released, can't be lock again

→ Growing phase is compulsory
It is important to put a lock

→ shrinking = optional

Conflict serializable 2PL Schedules

⇒ Each 2PL Schedules are conflict serializable, but not vice versa

Conflict serializable 2PL Schedules

Ex = 3

T ₁	T ₂
S(A)	
R(A)	
	X(A)
	R(A)
	W(A)
X(B)	
R(B)	
W(B)	
U(B)	
U(A)	
	U(A)

not allowed without
unlock(A)

∴ Not in 2PL

In 2PL

⇒ Exclusive lock Allow only after unlock data item

⇒ shared lock Allow without unlock data item

⇒ After Release lock on any data item within
that Xⁿ, Not allowed to put lock
again on that data
item
on that Xⁿ.

check whether schedule is in 2PL or not?

Ex-2

	<u>T₁</u>	<u>T₂</u>
growing phase	R(A)	
	R(A)	
	W(A)	
	S(B)	
	R(B)	
		S = shared lock x = Exclusive lock U = unlock
		Allowed S(B) } growing phase R(B) } U(B) } shrinking phase
shrinking phase	U(B)	
	U(A)	

→ only shared lock allowed to hold by other x
without unlock

It is in 2PL

Ex-2

	<u>T₁</u>	<u>T₂</u>
	S(A)	
	R(A)	
		S(B)
		R(B)
		U(B)
Exclusive lock on B after unlock allowed	X(B)	
	R(B)	
	W(B)	
	U(B)	
		S(B)
		R(B)
		U(B)

growing phase

shared on diff data item
∴ Allowed

shrinking

(growing phase)

Once again try to put lock not allowed

only no growing phase allowed after shrinking phase

∴ not in 2PL

* Problems with 2PL

never mentioned about commit

① 2PL doesn't ensure recoverability.

② Cascading Rollback

③ Deadlock

	T_1	T_2	
	X(A)	X(B)	
	!	!	
	!	!	
T_1 not Release A as it is known, after release it can't put back again	wait for B	wait for A	T_2 not Release B as it is known it can't put back again
	!	!	
	∞	∞	

Deadlock
↗

→ Both T_1 & T_2 are waiting for each other's data items.

~~Due to~~Due to hold & wait condⁿ

* Types of 2PL

Also generate cascades schedules
if generate conflict serializable schedule
which are recoverable

① strict 2PL $\Rightarrow$ Follow 2PL + All Exclusive lock
should unlock after commit.
 $\hookrightarrow$ Deadlock possible here.

② Rigorous 2PL $\Rightarrow$ Follow 2PL + All lock (S & X) should
unlock after commit operations.
 $\hookrightarrow$ generate $\left\{ \begin{array}{l} \text{Conflict serializability} \\ \text{Cascades Recoverable} \end{array} \right.$

③ Conservative 2PL

$\hookrightarrow$ No deadlock
 $\hookrightarrow$ perform any operation only on free data items.
(R/W)

strict
2PL

Rigorous 2PL

Ex: $\Rightarrow$ check 2PL, start 2PL or rigorous 2PL?

	T ₁	T ₂	
	SCA)		
	RCA)		
		SCB)	
		RCB)	
		X(C)	Exclusive lock
		R(C)	
		W(C)	
At 13		U(C) ←	unlock before commit
2PL		U(B)	Violate start 2PL
As lock obtained		C2 ←	$\therefore$ Not start 2PL & not
after unlock	SCC)		Rigorous 2PL as well,
	RC(C)		
	C1		

Ex: $\Rightarrow$	T ₁	T ₂	
	SCA)		
	RCA)		
		SCB)	
		RCB)	
		X(C)	At 13 start 2PL as unlock X(C) only after commit
		R(C)	
		W(C)	
At 13 2PL		U(B) ←	
		Commit	
	SCC) ←	U(C) ←	
	RC(C)		
	Commit		

Not rigorous 2PL, as SCB) not unlock after commit.

Ex: check whether scheduling has ^{start} 2PL and ^{Response} Response 2PL?

T ₁	T ₂
SCA)	
RCA)	
	SCB)
	RCB)
	XCC)
	RCC)
	W(C)
	UCC)
	UCB)
SCC)	
RCC)	
C1	
	C2

checking cond for 2PL

① check any data item

Not lock again after unlock within one Xⁿ.

② Another Xⁿ put lock on same data item only after unlock by previous Xⁿ (other)

③ follow table's rule

S	S	X
X	not	Not

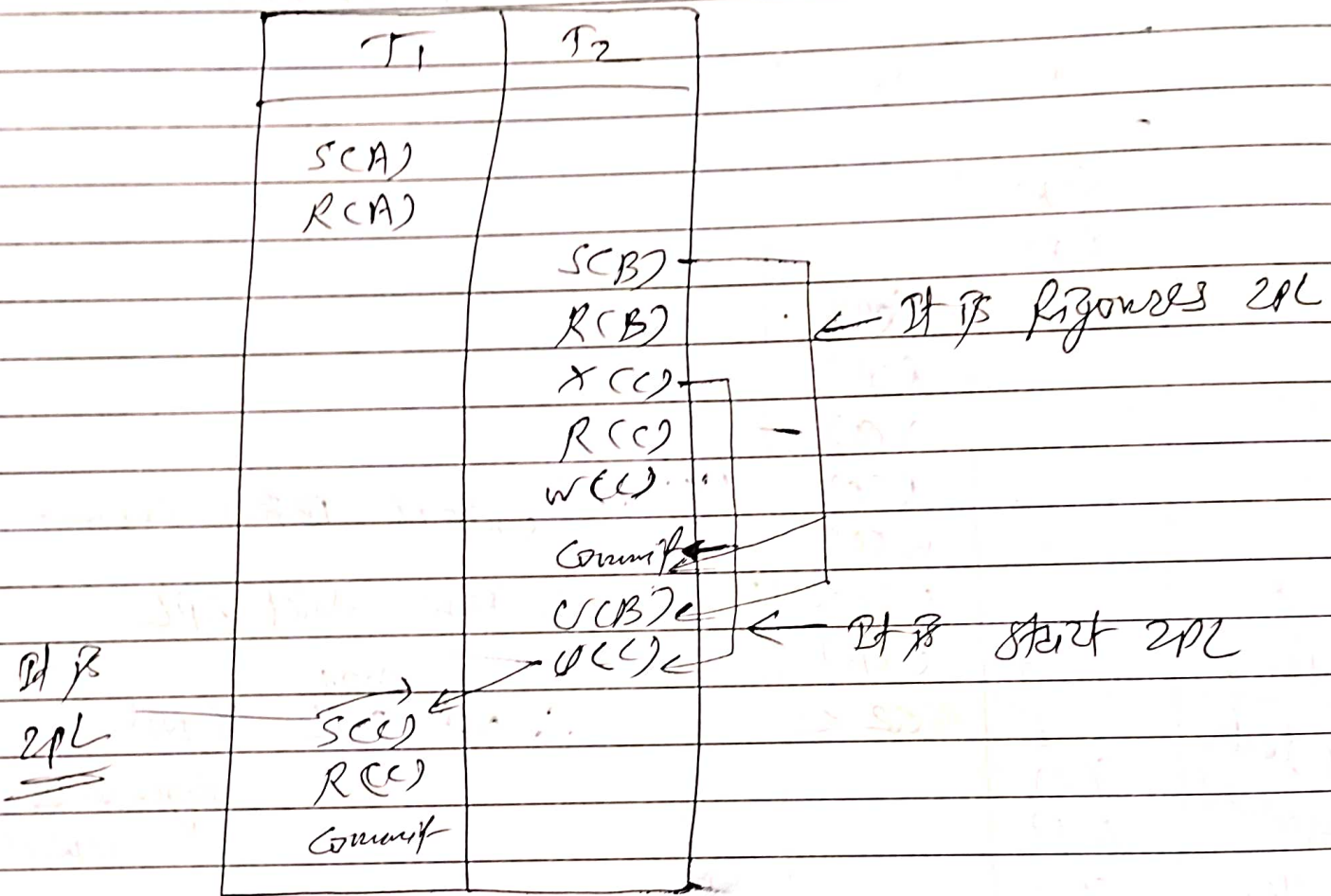
→ Not start 2PL
as unlock should done only after Commit

→ here unlock first, then Commit

∴ violate 2PL condⁿ start

As not start 2PL ∴ not Response 2PL

Ex :-> check 2PL, start 2PL & Rigorous 2PL



Note $\Rightarrow$ For Recoverability, it is important to mention about where & when to commit.

* Conservative 2PL → not so frequently used

↳ It Eliminate Deadlock.

↳ By Rule, check first all data items are free or not,

if all data items are free then only allocated to Transaction

→ data items not half-half or one one given to each T^n ,

And all data items given to one Transaction for it's complete work then all data items given to other T^n .

→

T_1	T_2
$W(A)$ $X(A)$ $X(B)$ Commit $U(A)$ $U(B)$	$A = B = \text{Free}$ $A = B = \text{Free}$ $X(A)$ $X(B)$ Commit $U(A)$ $U(B)$

→ No hold & wait condn secured here
 ∴ No deadlock.

* Time Stamp Ordering P/t

- Guarantee of serializability
 - Not provide guarantee of Recoverability
 - Free from Deadlock
 - Cascade Rollback is possible
- As not mentioned anything about when do Commit.

Algorithm

Time Stamp

Rule ① $TS(T_i) \geq WTS(x)$ ← write timestamp

then operation Executed

$$\hookrightarrow RTS = \max(RTS, TS(T_i))$$

② $TS(T_i) \geq RTS(x)$ ← read Time Stamp

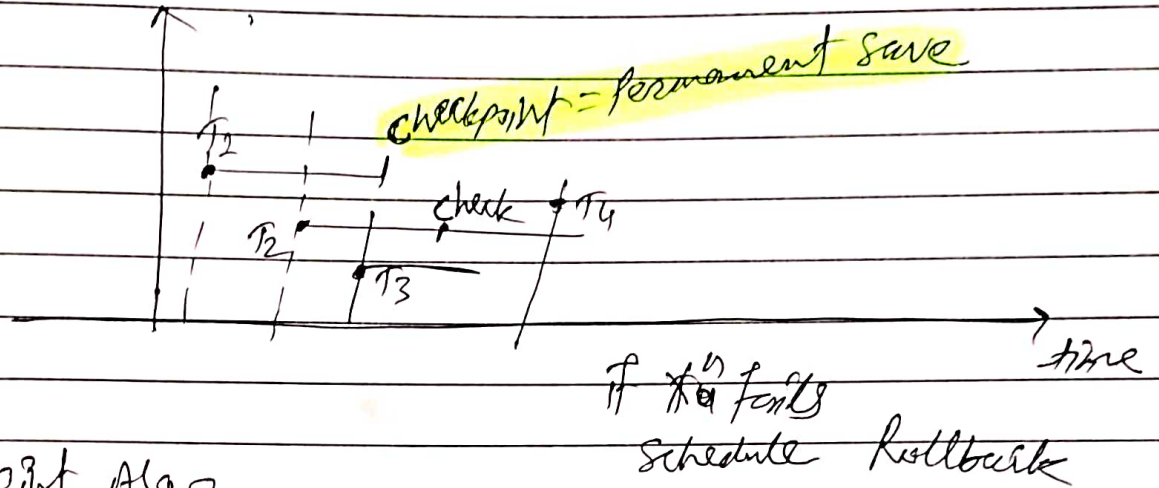
and

$$TS(T_i) \geq WTS(x)$$

then operation Executed,

$$\hookrightarrow WTS(x) = TS(T_i)$$

* checkpoint



checkpoint Algo

- make list of Undo ()
Redo ()
- X^n which committed before checkpoint need not perform any ~~op~~ previous ops again
- Such X^n committed saved permanently $\therefore$ no need for Undo & Redo list
- All X^n ^{committed} after checkpoint, put on Redo(list)
- All X^n ~~committed~~ ^{uncommitted} before & after checkpoint put on ~~Redo~~ ^{Undo} list.

Allow conflict serializability $\rightarrow$ Recoverability ensure

$\uparrow$
commit $\rightarrow$ No Cascade Rollback

Page No. _____

Date: ____/____/____

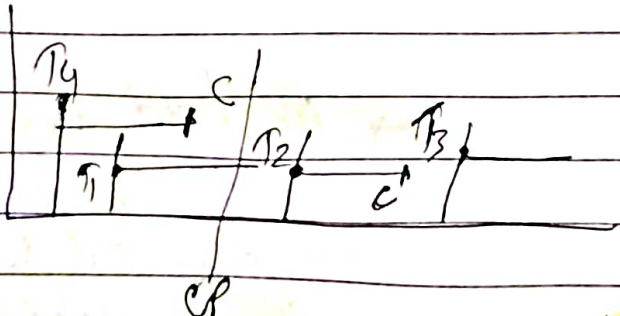
* 2.2.4 Time stamp ordering pth

- $\rightarrow$ Variation of Time ordering pth
- $\rightarrow$ Ensure Recoverability & Conflict serializability
- $\rightarrow$ R/W delayed & ^{Allowed} only after Commit & Abort.

* Thomas ^{waste} Right Rule

- $\rightarrow$ Not ensure conflict serializability
- $\rightarrow$ Gives guarantee of view serializability

Ex: check point list $\leftarrow$ Undo Redo



Undo (T₁, T₃)
Redo (T₂)

Redo = contain xⁿ commit after checkpoint

undo = contain All ^{xⁿ} before checkpoint.

not in Undo & Redo as
T₄ = committed before checkpoint
- not part of any list